MCT 442: Advanced Robotics Analysis and Control

3-DOF RRS Delta-Type Parallel Robot

Kinematics, Singularity Analysis & MATLAB Toolkit

Project: 3-DOF Parallel Robot (Delta-Type)

Milestone 1 — MATLAB Simulation Toolkit

Spring 2026

Table of Contents

1. Introduction and Background

1.1 What is a Delta Robot?

The Delta robot is a class of parallel kinematic machine (PKM) invented by Reymond Clavel at EPFL in 1985 and patented in 1988. Unlike serial robots where joints are arranged in a chain from base to end-effector, the Delta robot uses three identical kinematic chains connecting a fixed base to a moving platform. The three chains work in parallel to position the end-effector in three-dimensional space while constraining its orientation — the platform always remains parallel to the base. This makes the Delta robot a 3-DOF purely translational device.

The specific variant used in this project is the RRS type: each kinematic chain consists of one active Revolute joint (the motor), one passive Revolute joint (the parallelogram mechanism), and one passive Spherical joint at the platform attachment. The parallelogram structure of the forearm links is what maintains the platform orientation fixed throughout all motions.

1.2 Why Parallel Robots?

Parallel robots offer several advantages over serial robots that make them particularly attractive for pick-and-place and precision applications:

- High speed and acceleration: all three motors are mounted at the base, minimising the moving mass. This is why Delta robots are the dominant robot design in food packaging and pharmaceutical sorting, reaching accelerations of 50 g.
- High stiffness: the load is shared across three independent chains, giving far greater structural stiffness than a comparable serial robot.
- Good repeatability: the closed-loop kinematics constrain the platform tightly.
- Compact footprint: ideal for over-conveyor installations.

The main disadvantages are a smaller and more complex-shaped workspace compared to serial robots, and the need to solve more involved kinematic equations (both forward and inverse kinematics have non-trivial closed-form solutions).

1.3 Coordinate System Convention

All five MATLAB files use a unified coordinate system defined here. It is essential to understand this convention before using any of the functions.

Coordinate Frame Convention

Origin: Centre of the fixed base platform.
+X axis: Aligned with Chain 1 (azimuth $\phi_1 = 0$ deg).
+Y axis: Perpendicular to X, in the base plane.
+Z axis: Pointing DOWNWARD (into the workspace below the base).

Consequence: Reachable workspace has $z > 0$ (positive z = below base).
Joint angle: $\theta_i = 0$ when upper arm is horizontal.
$\theta_i > 0$ when elbow moves below base plane (normal operation).
Chain azimuths: ϕ_1=0 deg, ϕ_2=120 deg, ϕ_3=240 deg.

Note: some textbooks use +Z pointing upward, in which case the workspace has z < 0. If your physical robot or URDF uses a different Z convention, negate the z-component when passing positions to delta_IK() and negate the z output of delta_FK().

1.4 File Overview

The toolkit consists of five MATLAB function files. The dependency graph flows strictly downward — no circular dependencies exist.

MATLAB File	Primary Function	Depends On
delta_params.m	Returns robot parameter struct	(standalone)
delta_IK.m	Closed-form IK; returns joint angles	delta_params.m
delta_FK.m	Sphere-intersection FK; returns EE position	delta_params.m
delta_workspace.m	3-D workspace scan and visualisation	delta_IK.m, delta_params.m
delta_singularity.m	Jacobian + singularity flags	delta_FK.m, delta_params.m

2. delta_params.m

2.1 Purpose

delta_params.m is the single authoritative source of all robot geometric parameters and joint limits. Every other function in the toolkit calls delta_params() to obtain a consistent parameter set. Editing robot dimensions in one place (this file) automatically propagates to the entire toolkit.

2.2 Parameters Defined

Symbol	Description	Typical Value	Units
Rb	Base platform circumradius (motor joint triangle)	0.200	m
Rp	Moving platform circumradius (EE attachment triangle)	0.060	m
L1	Upper arm (proximal link) length	0.200	m
L2	Forearm (distal parallelogram) length	0.400	m
φi	Azimuth angles: 0°, 120°, 240°	0, 2π/3, 4π/3	rad
θ_min	Joint angle lower limit (most retracted)	-20°	deg
θ_max	Joint angle upper limit (most extended)	90°	deg

The file also computes two derived convenience fields: params.f_base (base triangle side length = Rb*sqrt(3)) and params.e_platform (platform triangle side length = Rp*sqrt(3)).

2.3 Physical Meaning of Robot Dimensions

The four fundamental geometric parameters relate to the physical robot as follows:

- Rb: measure the distance from the centre of the base plate to any one of the three motor-shaft centres. This is the circumradius of the equilateral triangle traced by the three motor joints.
- Rp: measure the distance from the centre of the moving platform to any one of the three spherical-joint centres on the platform.
- L1: the length from the motor-shaft centre to the proximal spherical joint of the parallelogram arm (the rigid upper arm that rotates when the motor turns).
- L2: the effective length of the parallelogram forearm, from the proximal spherical joint to the platform spherical joint.

2.4 How to Adapt to Your Robot

After designing or measuring your physical robot, replace the four numerical values in the parameters section of delta_params.m. The remainder of the toolkit requires no changes.

```
% Example: customising for your specific hardware
params.Rb = 0.185;    % measured from centre of base to motor shaft [m]
params.Rp = 0.055;    % measured from centre of platform to spherical joint [m]
params.L1 = 0.195;    % upper arm length [m]
params.L2 = 0.385;    % forearm effective length [m]
params.th_min = deg2rad(-15);  % from your servo/DC motor data sheet
params.th_max = deg2rad( 85);
```

Also update params.ws_lim after running delta_workspace.m to tighten the bounding box to the actual reachable volume.

2.5 Usage Example

```
% Load parameters
params = delta_params();

% Access individual fields
fprintf('Upper arm length L1 = %.3f m\n', params.L1);
fprintf('Maximum reach = %.3f m\n', params.L1 + params.L2);
```

3. delta_IK.m

3.1 Purpose

delta_IK.m computes the three joint angles (θ_1, θ_2, θ_3) for a given end-effector Cartesian position p = (x, y, z). This is the problem you solve every time you command the robot to go to a specific position: you know where you want the tool to be, but the motors require joint-angle targets.

3.2 Mathematical Derivation

3.2.1 Geometric Setup

For kinematic chain i (i = 1, 2, 3), three points are defined:

$$B_i = Rb * [\cos(\phi_i), \sin(\phi_i), \quad 0]' \quad (base\ joint\ centre, fixed)$$

$$E_i = [(Rb + L1 * cos(\theta_i)) * cos(\phi_i), (Rb + L1 * cos(\theta_i)) * sin(\phi_i), \\ L1 * \sin(theta\_i)]' \quad (elbow, depends\ on\ theta\_i)$$

$$P_i = p + Rp * [cos(\phi_i), sin(\phi_i), 0]' \quad (platform\ attachment, depends\ on\ p)$$

The elbow E_i is constrained to lie in the vertical plane that contains B_i and is parallel to the z-axis. This is because the upper arm is driven by a revolute joint whose axis is horizontal and tangent to the base circle at B_i.

3.2.2 Kinematic Constraint

The forearm length L2 is constant. Therefore:

$$|E_i - P_i|^2 = L2^2 \quad \text{... (Constraint equation for chain i)}$$

3.2.3 Reduction to a Single Unknown

Defining the in-plane projection s_i and the out-of-plane component t_i of the end-effector position:

$$s_i = x * cos(\phi_i) + y * sin(\phi_i)$$
$$t_i = -x * sin(\phi_i) + y * cos(\phi_i)$$
$$a_i = (Rb - Rp) - s_i$$

Substituting E_i and P_i into the constraint and expanding:

$$a_i * cos(\theta_i) - z * sin(\theta_i) = C_i$$

$$where \ C_i = \frac{L2^2 - L1^2 - a_i^2 - t_i^2 - z^2}{2 * L1}$$

This is a single-variable trigonometric equation in θ_i. Note that t_i appears only in C_i — the perpendicular (out-of-plane) component of the end-effector displacement enters as a correction to the effective reach.

3.2.4 Closed-Form Solution

The equation $A * cos(\theta) + B * sin(\theta) = C$ (with A = a_i , B = -z) has the standard solution using the auxiliary angle psi = atan2(B, A):

$$\sqrt{(A^2 + B^2)} * cos(\theta - \tan^{-1}(-\frac{z}{a_i})) = C$$

$$\theta_i = \tan^{-1}(-\frac{z}{a_i}) + \cos^{-1}\left(\frac{C_i}{\sqrt{(a_i^2 + z^2)}}\right)$$

The + sign selects the physically meaningful elbow-up branch. The - sign corresponds to the elbow-down (self-intersecting) configuration where the upper arm crosses above the base platform, which is mechanically unreachable in a standard delta robot.

3.2.5 Feasibility Condition

A real solution exists for chain i if and only if:

$$|C_i / sqrt(a_i^2 + z^2)| <= 1$$

If this condition is violated, the point p is outside the reachable shell of chain i (too far or too close for that chain's geometry). The function returns valid = false and θ(i) = NaN.

3.3 Function Signature and Outputs

```
[θ, valid]          = delta_IK(params, p)
[θ, valid, err_msg] = delta_IK(params, p)

% INPUTS
%   params : struct from delta_params()
%   p      : 3x1 desired EE position [x; y; z]  in metres

% OUTPUTS
%   θ    : 3x1 joint angles [θ1; θ2; θ3]  in radians
%            (NaN for any chain with no solution)
%   valid   : true if solution exists AND all joint limits satisfied
%   err_msg : string describing failure reason (empty if valid)
```

3.4 Usage Examples

```
params = delta_params();

% Example 1: Point at the centre of the workspace
p = [0; 0; 0.35];
[θ, valid] = delta_IK(params, p);
if valid
    fprintf('θ = [%.2f, %.2f, %.2f] deg\n', ...
            rad2deg(θ(1)), rad2deg(θ(2)), rad2deg(θ(3)));
else
    fprintf('Point unreachable.\n');
end

% Example 2: Validate IK by round-tripping through FK
[θ, ok] = delta_IK(params, [0.05; -0.03; 0.30]);
if ok
    [p_fk, ~] = delta_FK(params, θ);
    e = norm(p_fk - [0.05; -0.03; 0.30]);
    fprintf('IK/FK round-trip error: %.6f m\n', e);
end
```

3.5 Edge Cases and Warnings

- If p lies exactly on the z-axis of chain i (a_i = 0 AND z = 0), the denominator sqrt(a_i^2 + z^2) = 0 and the solution is degenerate. The function returns valid = false.
- If |C_i| > sqrt(a_i^2 + z^2) for any chain, no real θ_i exists. The point is outside the annular reach of that chain.
- Joint limit violations are checked after computing θ. A point geometrically reachable but outside joint limits returns valid = false.

4. delta_FK.m

4.1 Purpose

delta_FK.m computes the end-effector Cartesian position p = (x, y, z) given the three joint angles (θ_1, θ_2, θ_3). This is used when the robot's encoder readings are available and you want to know where the end-effector actually is in space — the direct kinematics problem.

Unlike serial robots where FK has a simple chain-multiplication formula, the delta robot's FK requires solving a system of three simultaneous sphere equations. delta_FK.m implements an exact, efficient closed-form solution via sphere-intersection and quadratic formula.

4.2 Mathematical Derivation

4.2.1 Shifted Elbow Centres

For each chain i, given θ_i, the elbow position E_i is fully determined:

$$E_i = [(Rb + L1 * cos(\theta_i)) * cos(\phi_i), \\ (Rb + L1 * cos(\theta_i)) * sin(\phi_i), \\ L1 * sin(\theta_i)]'$$

The kinematic constraint |E_i - P_i|^2 = L2^2 with P_i = p + Rp*[cos(ϕ_i), sin(ϕ_i), 0]' can be rewritten by defining shifted elbow centres:

$$E_i' = E_i - Rp * [cos(\phi\_i), sin(\phi\_i), 0]' \\ = [(Rb - Rp + L1 * cos(\theta\_i)) * cos(\phi\_i), \\ (Rb - Rp + L1 * cos(\theta\_i)) * sin(\phi\_i), \\ L1 * sin(\theta\_i)]'$$

Now the three constraints become three sphere equations in p:

$$|p - E_1'|^2 = L2^2 \quad \dots(S1) \\ |p - E_2'|^2 = L2^2 \quad \dots(S2) \\ |p - E_3'|^2 = L2^2 \quad \dots(S3)$$

4.2.2 Linearization by Sphere Subtraction

Subtracting (S1) from (S2) and from (S3) eliminates the quadratic term p'p and yields two linear equations:

$$2 * (E_k' - E_1')' * p = |E_k'|\text{^}2 - |E_1'|\text{^}2 \quad for\ k = 2,3$$

$$Writing\ as:\ A * p = b \\ where\ A(k-1,:) = 2 * (E_k' - E_1')' \\ b(k-1) = |E_k'|\text{^}2 - |E_1'|\text{^}2$$

4.2.3 Parameterisation in z

The 2x3 system A*p = b is under-determined (2 equations, 3 unknowns). Partitioning A = [A_xy | A_z]:

$$\begin{aligned} A_{xy} * [x;\, y] &= b - A_z * z \\ [x;\, y] &= A_{xy}^{\{-1\}} * (b - A_z * z) \\ &= k_{const} + k_{slope} * z \end{aligned}$$

x and y are now affine functions of z. Substituting back into sphere (S1) gives a quadratic in z:

$$\begin{aligned} (sx^2 + sy^2 + sz^2) * z^2 + 2 * (dx0 * sx + dy0 * sy + dz0 * sz) * z \\ + (dx0^2 + dy0^2 + dz0^2 - L2^2) = 0 \end{aligned}$$

where [dx0, dy0, dz0] is the constant part and [sx, sy, sz=1] the z-slope of (p - E_1').

4.2.4 Root Selection

The quadratic has two roots. Since +Z points downward, the root with the LARGER z value corresponds to the deeper (workspace-side) solution — the physically relevant configuration for a delta robot operating below its base. The smaller root represents a mirror image above the base, which is mechanically unreachable.

4.3 Function Signature

```
[p, valid]          = delta_FK(params, θ)
[p, valid, err_msg] = delta_FK(params, θ)

% INPUTS
%   params : struct from delta_params()
%   θ  : 3x1 joint angles [θ1; θ2; θ3]  in radians

% OUTPUTS
%   p       : 3x1 EE position [x; y; z]  in metres
%   valid   : true if a real unique solution exists
%   err_msg : string describing failure (empty if valid)
```

4.4 Usage Examples

```
params = delta_params();

% Compute FK from known joint angles
θ = deg2rad([30; 30; 30]);
[p, valid] = delta_FK(params, θ);
fprintf('EE position: [%.4f, %.4f, %.4f] m\n', p(1), p(2), p(3));

% Validate FK / IK consistency (IK/FK round-trip)
p_target = [0.05; 0.03; 0.32];
[θ_ik, ok_ik] = delta_IK(params, p_target);
[p_fk,     ok_fk] = delta_FK(params, θ_ik);
e_cons = norm(p_fk - p_target);
fprintf('IK/FK consistency error: %.2e m (should be < 1e-10)\n', e_cons);
```

5. delta_workspace.m

5.1 Purpose

delta_workspace.m characterises the reachable workspace of the delta robot by exhaustive sampling of the Cartesian bounding box defined in params.ws_lim. The function provides both a 3-D visual plot and a programmatic workspace membership test.

5.2 What is the Delta Robot Workspace?

The reachable workspace is the set of all end-effector positions p = (x, y, z) for which there exists a valid IK solution with all joint angles within their hardware limits. For the delta robot this workspace has the following characteristic shape:

- Overall shape: a toroidal (doughnut-like) shell with a roughly circular cross-section, truncated at the top and bottom by joint limits.
- Outer boundary: defined by the maximum arm extension. Points farther than approximately (L1 + L2) - (Rb - Rp) from the z-axis at the maximum-reach angle are unreachable.
- Inner boundary (dead zone): exists if L1 and L2 are similar in length. Points very close to the z-axis and very high in z may be unreachable because the arm cannot fold enough.
- Vertical extent: controlled by the joint angle limits θ_min and θ_max. Tighter limits produce a shallower workspace.
- Symmetry: the workspace is rotationally symmetric about the z-axis for a symmetric delta robot (all three chains identical).

5.3 Two Operation Modes

Mode A — Full 3-D Scan

Samples N^3 points on a regular grid and tests each with delta_IK(). Outputs a point cloud and generates a 3-D scatter plot coloured by depth (z). A 2-D XZ cross-section is also shown in the corner of the figure for reference.

```
params = delta_params();

% Basic scan at default resolution (25^3 = 15625 points)
[points, in_ws] = delta_workspace(params);

% Higher resolution scan (slower, ~2 min at res=40)
[points, in_ws] = delta_workspace(params, 'res', 40);

% Suppress the plot
[points, in_ws] = delta_workspace(params, 'res', 30, 'plot', false);

% Extract reachable points
reach = points(in_ws, :);
fprintf('Reachable: %d / %d points\n', sum(in_ws), numel(in_ws));
```

Mode B — Single-Point Membership Test

For trajectory planning and motion control, use the fast single-point check (no plot, no full scan):

```
params = delta_params();

p_candidate = [0.05; -0.02; 0.38];
in_ws = delta_workspace(params, 'check', p_candidate);

if in_ws
    disp('Safe to move to this point.');
else
    disp('STOP: point is outside workspace!');
end
```

5.4 Using Workspace Results to Update params.ws_lim

After running the full scan, update the bounding box in delta_params.m with the actual observed limits. This speeds up future IK calls because the boundary pre-check rejects impossible points early:

```
params = delta_params();
[pts, in_ws] = delta_workspace(params, 'res', 35, 'plot', false);
reach = pts(in_ws, :);

% Print tighter bounding box
fprintf('x: [%.4f, %.4f]\n', min(reach(:,1)), max(reach(:,1)));
fprintf('y: [%.4f, %.4f]\n', min(reach(:,2)), max(reach(:,2)));
fprintf('z: [%.4f, %.4f]\n', min(reach(:,3)), max(reach(:,3)));

% Then update params.ws_lim in delta_params.m with these values
% (add a small safety margin of ~0.01 m)
```

6. delta_singularity.m

6.1 Purpose

delta_singularity.m computes the velocity Jacobian matrices and detects whether the robot is at or near a singular configuration. Singularities are critical to understand and avoid because at a singularity the robot loses the ability to control its end-effector in certain directions, becomes structurally compliant (collapses under load), or requires infinite joint forces to produce finite end-effector forces.

6.2 Velocity Kinematics Derivation

The velocity relationship is obtained by differentiating the three kinematic constraints |E_i - P_i|^2 = L2^2 with respect to time:

$$2 * (E_i - P_i)' * (\frac{dE_i}{dt} - \frac{dp}{dt}) = 0 \qquad for\ i = 1,2,3$$

Since E_i depends only on θ_i, and the platform is purely translational (dp/dt = p_dot):

$$\frac{dE_i}{dt} = (\frac{dE_i}{d\theta_i}) * \theta\_dot_i = v_i * \theta\_dot_i$$

$$v_i = L1 * [-sin(\theta_i) * cos(\phi_i), \quad -sin(\theta_i) * sin(\phi_i), \quad cos(\theta_i)]'$$

Defining the forearm unit vector n_i = (E_i - P_i) / L2, the constraint for chain i becomes:

$$n_i' * v_i * \theta\_dot_i = n_i' * p\_dot$$

Stacking all three chains into matrix form:

$$Jq * \theta\_dot = Jx * p\_dot$$

$$Jq = diag([n_1' * v_1, \quad n_2' * v_2, \quad n_3' * v_3]) \quad (3x3\ diagonal)$$
$$Jx = [n_1'; \ n_2'; \ n_3'] \quad (3x3, rows = forearm\ unit\ vectors)$$

The full velocity mappings are therefore:

$$p_{dot} = Jx^{-1} * Jq * \theta_{dot} \quad (FK\ velocity: joints -> Cartesian)$$
$$\theta_{dot} = Jq^{-1} * Jx * p_{dot} \quad (IK\ velocity: Cartesian -> joints)$$

6.3 Singularity Types

Type	Also called	Condition	Cause	Risk
Type 1	FK / Direct / Parallel	det(Jx) = 0	Forearms coplanar	Loss of stiffness; DANGEROUS
Type 2	IK / Inverse / Serial	any Jq(i,i) = 0	Arm fully extended/folded	Workspace boundary
Type 3	Combined / Mixed	Both above	Special symmetry	Both effects

6.3.1 Type-1 Singularity: det(Jx) = 0 (MOST DANGEROUS)

Jx is the matrix whose rows are the three forearm unit vectors. det(Jx) = 0 means these three vectors are linearly dependent — they all lie in a common plane. When this happens:

- The robot gains one or more extra degrees of freedom even with all joints locked.
- The platform can undergo an infinitesimal motion without moving any joint. This motion is uncontrollable.
- Joint torques required to resist external forces in certain directions become theoretically infinite.
- Structural stiffness collapses in the direction perpendicular to the common plane of the forearms.

In practice, Type-1 singularities occur at two regions: when all three forearms point vertically (at the very top of the workspace, extreme high-z limit) and when all three forearms are in the same horizontal plane (arms extended outward at the same height, outer boundary of workspace).

6.3.2 Type-2 Singularity: det(Jq) = 0

Jq is diagonal, so det(Jq) = 0 iff at least one diagonal entry Jq(i,i) = n_i . v_i = 0. For chain i, this means the forearm n_i is perpendicular to the elbow tangent v_i. Geometrically, this means the upper arm and forearm of chain i are collinear:

- Arm fully extended: the forearm points directly away from the elbow. Chain i cannot exert any force along its extended direction.
- Arm fully folded back: the forearm doubles back onto the upper arm. Occurs at θ_i = θ_min.

Type-2 singularities define the boundaries of the workspace for each individual chain. They are less dangerous than Type-1 because only one chain is affected at a time (in normal operation), but they still produce very large joint velocities when attempting to move through them.

6.4 Condition Number as a Proximity Metric

The condition number of Jx, defined as cond(Jx) = sigma_max(Jx) / sigma_min(Jx) (ratio of largest to smallest singular values), measures how close the robot is to a Type-1 singularity:

- cond(Jx) = 1: perfectly isotropic configuration (equal force transmission in all directions, ideal).
- cond(Jx) < 10: good configuration, well away from singularity.
- cond(Jx) = 10-50: moderate proximity; use caution in trajectory planning.
- cond(Jx) > 50: close to singularity; trajectory should be rerouted.
- cond(Jx) = infinity (or NaN): singularity reached.

6.5 Function Signature

```
[Jx, Jq, type1, type2, info] = delta_singularity(params, θ)

% INPUTS
%   params : struct from delta_params()
%   θ  : 3x1 joint angles in radians

% OUTPUTS
%   Jx     : 3x3 Cartesian Jacobian (rows = forearm unit vectors)
%   Jq     : 3x3 diagonal joint Jacobian
%   type1  : logical, true if |det(Jx)| < SING_TOL
%   type2  : logical, true if any |Jq(i,i)| < SING_TOL
%   info   : struct with extended diagnostics (see code for fields)
```

6.6 Usage Examples

```
params = delta_params();

% Check singularity at a given configuration
θ = deg2rad([30; 30; 30]);
[Jx, Jq, t1, t2, info] = delta_singularity(params, θ);

% Use condition number to colour-code a trajectory
N = 50;  cond_vals = zeros(1, N);
for k = 1:N
    p_k = [0.05*cos(2*pi*k/N); 0.05*sin(2*pi*k/N); 0.35];
    [th_k, ok] = delta_IK(params, p_k);
    if ok
        [~,~,~,~, inf_k] = delta_singularity(params, th_k);
        cond_vals(k) = inf_k.cond_Jx;
    end
end
plot(cond_vals); yline(50,'r--','Singularity warning');
xlabel('Trajectory point'); ylabel('cond(Jx)');
```

9. References

The derivations and implementations in this toolkit are based on the following primary sources:

[1] Clavel, R. (1988). Delta, a fast robot with parallel geometry. Proceedings of the International Symposium on Industrial Robots (ISIR), Lausanne, Switzerland, pp. 91-100.

Original invention paper. Describes the Delta robot concept, geometric constraints, and IK principle.

[2] Merlet, J.-P. (2006). Parallel Robots, 2nd ed. Springer, Netherlands.

Comprehensive textbook on parallel mechanisms. Chapters 3-4 cover Delta IK/FK derivations; Chapter 6 covers workspace analysis and singularities (Types 1, 2, combined).

[3] Gosselin, C. & Angeles, J. (1990). Singularity Analysis of Closed-Loop Kinematic Chains. IEEE Transactions on Robotics and Automation, 6(3), 281-290.

Foundational paper defining Type-1 and Type-2 singularities for parallel mechanisms. The Jx/Jq Jacobian framework used in delta_singularity.m is introduced here.

[4] Tsai, L.-W. (1999). Robot Analysis: The Mechanics of Serial and Parallel Manipulators. Wiley, New York. Chapter 7.

Detailed treatment of the Delta robot forward and inverse kinematics using the sphere-intersection method (used in delta_FK.m).

[5] Huang, Z., Liu, J. & Zeng, D. (2013). Theory of Parallel Mechanisms. Springer.

Chapter 4 provides velocity analysis and Jacobian derivations for Delta-type robots. Source for the per-chain tangent vector v_i derivation.

[6] Stock, M. & Miller, K. (2003). Optimal Kinematic Design of Spatial Parallel Manipulators. ASME Journal of Mechanical Design, 125(2), 292-301.

Covers workspace optimisation for delta robots including effects of Rb, Rp, L1, L2 on workspace shape.

[7] Siciliano, B., Sciavicco, L., Villani, L. & Oriolo, G. (2009). Robotics: Modelling, Planning and Control. Springer.

General robotics textbook used for coordinate frame conventions, Jacobian theory, and singularity definitions.

[8] Zlatanov, D., Fenton, R.G. & Benhabib, B. (1994). Singularity Analysis of Mechanisms and Robots via a Velocity-Equation Model of the Instantaneous Kinematics. Proceedings of IEEE ICRA, pp. 986-991.

Extended singularity taxonomy (Type 3 combined singularities). Referenced in delta_singularity.m comments.